Chapter 7: Arrays

COMP 110
Single-dimensional arrays
Examples, discussion, and practice

Learning goals

Create, index, and traverse arrays
Process and copy array elements
Explain array references in method calls
Search and sort arrays
Use varargs and command-line arguments

Opening problem: above the average

Read 100 numbers and compute their average.
Then count how many exceed that average.

Discussion: Why does one running total
not give us everything we need?

Declaration, creation, and assignment

```
double[] scores;
scores = new double[3];
scores[0] = 82.5;
scores[1] = 90.0;
System.out.println(scores.length);
System.out.println(scores[0]);
System.out.println(scores[2]);
```

The variable holds a reference. The array holds the elements.

Default element values

```
int[] counts = new int[2];
boolean[] flags = new boolean[2];
char[] letters = new char[2];
String[] names = new String[2];
System.out.println(counts[0]);
System.out.println(flags[0]);
System.out.println((int) letters[0]);
System.out.println(names[0]);
```

Numeric: 0. Boolean: false. Character: \u0000. Reference: null.

Array initializers

```
double[] values = {1.9, 2.9, 3.4, 3.5};
System.out.println(values.length);
values = new double[]{8.0, 9.0};
System.out.println(Arrays.toString(values));
// values = {8.0, 9.0}; // Does not compile.
```

The shorthand initializer belongs in a declaration.

Indexes and array bounds

```
int[] values = {10, 20, 30};
int last = values.length - 1;
values[1] += 5;
System.out.println(values[0]);
System.out.println(values[last]);
System.out.println(values[1]);
// System.out.println(values[3]); // Runtime error.
```

For length 3, valid indexes are 0 through 2.

Quiz 1: creation and bounds

Given int[] a = new int[4]:

1. What are a.length and a[3]?
2. Is index 4 valid?
3. What indexes can new int[0] use?
4. Does a.length() compile?

Tracing updates across an array

```
int[] values = new int[5];
for (int i = 1; i < values.length; i++) {
    values[i] = i + values[i - 1];
}
values[0] = values[1] + values[4];
System.out.println(Arrays.toString(values));
```

Prediction: write the array after each iteration.

Trace results

Initially: [0, 0, 0, 0, 0]
i = 1: [0, 1, 0, 0, 0]
i = 2: [0, 1, 3, 0, 0]
i = 3: [0, 1, 3, 6, 0]
i = 4: [0, 1, 3, 6, 10]
Final assignment: [11, 1, 3, 6, 10]

An indexed loop can update elements

```
int[] values = {4, 7, 2};
for (int i = 0; i < values.length; i++) {
    values[i] *= 2;
    System.out.println(i + ": " + values[i]);
}
```

The condition i < values.length protects the upper bound.

A for-each variable receives each value

```
int[] values = {4, 7, 2};
for (int value : values) {
    value *= 2;
}
System.out.println(Arrays.toString(values));
for (int value : values) {
    System.out.println(value);
}
```

Assigning to value leaves these int elements unchanged.

Discussion: choosing a loop

Which loop fits each task?

Print every value
Double every int element
Print elements in reverse order
Find the first matching index

Filling an array from input

```
Scanner input = new Scanner(System.in);
double[] values = new double[3];
for (int i = 0; i < values.length; i++) {
    values[i] = input.nextDouble();
}
System.out.println(Arrays.toString(values));
```

Example input: 2.5 4.0 7.5

Filling an array with random values

```
Random random = new Random(7);
int[] values = new int[5];
for (int i = 0; i < values.length; i++) {
    values[i] = random.nextInt(10);
}
System.out.println(Arrays.toString(values));
```

nextInt(10) produces a value from 0 through 9.

Total and average

```
double[] values = {10, 20, 30, 40};
double total = 0;
for (double value : values) {
    total += value;
}
double average = total / values.length;
System.out.println(total);
System.out.println(average);
```

Precondition: the array contains at least one value.

The first index of the maximum

```
static int firstMaximum(int[] values) {
    int best = 0;
    for (int i = 1; i < values.length; i++) {
        if (values[i] > values[best]) best = i;
    }
    return best;
}
```

Use > to preserve the first occurrence when values tie.

Quiz 2: array processing

1. Why use the first element to initialize a maximum?

2. For {8, 3, 8}, does >= keep the first maximum?

3. What should an average method do
   when its array is empty?

Case study: AnalyzeNumbers

```
Scanner input = new Scanner(System.in);
double[] values = new double[5];
for (int i = 0; i < values.length; i++) {
    values[i] = input.nextDouble();
}
double mean = average(values);
System.out.println("Average: " + mean);
System.out.println("Above: " + countAbove(values, mean));
```

Input 2 4 6 8 10 gives average 6.0 and above-count 2.

AnalyzeNumbers helper methods

```
static double average(double[] values) {
    double total = 0;
    for (double value : values) total += value;
    return total / values.length;
}

static int countAbove(double[] values, double mean) {
    int count = 0;
    for (double value : values) {
        if (value > mean) count++;
    }
    return count;
}
```

One pass computes the average. A second pass counts values.

Shuffling with Fisher-Yates

```
static void shuffle(int[] values, Random random) {
    for (int i = values.length - 1; i > 0; i--) {
        int j = random.nextInt(i + 1);
        int temp = values[i];
        values[i] = values[j];
        values[j] = temp;
    }
}
```

Choose j from 0 through i, then swap.

Rotating left by one position

```
static void rotateLeft(int[] values) {
    if (values.length < 2) return;
    int first = values[0];
    for (int i = 1; i < values.length; i++) {
        values[i - 1] = values[i];
    }
    values[values.length - 1] = first;
}
```

[1, 2, 3, 4] becomes [2, 3, 4, 1].

Rotating right by one position

```
static void rotateRight(int[] values) {
    if (values.length < 2) return;
    int last = values[values.length - 1];
    for (int i = values.length - 1; i > 0; i--) {
        values[i] = values[i - 1];
    }
    values[0] = last;
}
```

[1, 2, 3, 4] becomes [4, 1, 2, 3].

Discussion: preserving array values

Why do rotations save one endpoint first?

Why do the left and right rotations
copy in different directions?

How does a rotation differ from deleting
the first or last element?

Partially filled arrays

```
int[] values = new int[5];
int size = 0;
values[size++] = 10;
values[size++] = 20;
for (int i = 0; i < size; i++) {
    System.out.println(values[i]);
}
System.out.println("size=" + size);
System.out.println("capacity=" + values.length);
```

Capacity is values.length. Logical size counts used elements.

Deleting from a partially filled array

```
static int deleteAt(int[] values, int size, int index) {
    for (int i = index; i < size - 1; i++) {
        values[i] = values[i + 1];
    }
    values[size - 1] = 0;
    return size - 1;
}
```

Delete index 1 from [10, 20, 30, 40] to get [10, 30, 40].

Insertion into a sorted prefix

```
static int insertSorted(int[] values, int size, int key) {
    int i = size;
    while (i > 0 && values[i - 1] > key) {
        values[i] = values[i - 1];
        i--;
    }
    values[i] = key;
    return size + 1;
}
```

Shift larger elements right to make room for the new value.

Quiz 3: size and capacity

A buffer has length 8 and logical size 3.

1. Which indexes contain used entries?
2. Can you append one more value?
3. After deleting one entry, what are the logical size and physical length?

Case study: four cards from one deck

```
int[] deck = new int[52];
for (int i = 0; i < deck.length; i++) deck[i] = i;
ShuffleDemo.shuffle(deck, new Random(7));
for (int i = 0; i < 4; i++) {
    System.out.println(cardName(deck[i]));
}
```

Shuffling unique card IDs avoids drawing the same card twice.

Decoding a card ID

```
static String cardName(int card) {
    String[] suits = {"Spades", "Hearts", "Diamonds", "Clubs"};
    String[] ranks = {"Ace", "2", "3", "4", "5", "6", "7",
                      "8", "9", "10", "Jack", "Queen", "King"};
    return ranks[card % 13] + " of " + suits[card / 13];
}
```

Suit = card / 13. Rank = card % 13.

Assignment copies the reference

```
int[] first = {2, 3, 1};
int[] second = first;
second[0] = 99;
System.out.println(Arrays.toString(first));
System.out.println(first == second);
```

first and second refer to the same array.

Copying elements into a new array

```
int[] source = {2, 3, 1};
int[] target = new int[source.length];
for (int i = 0; i < source.length; i++) {
    target[i] = source[i];
}
target[0] = 99;
System.out.println(Arrays.toString(source));
System.out.println(Arrays.toString(target));
```

The new int array has independent element storage.

System.arraycopy and Arrays.copyOf

```
int[] source = {2, 3, 1, 5};
int[] target = new int[4];
System.arraycopy(source, 1, target, 0, 2);
int[] copy = Arrays.copyOf(source, source.length);
System.out.println(Arrays.toString(target));
System.out.println(Arrays.toString(copy));
```

arraycopy needs a destination. copyOf creates a new array.

Quiz 4: aliases and copies

Start with a = {2, 3, 1}.

1. After b = a and b[0] = 9, what is a[0]?
2. After b = Arrays.copyOf(a, a.length), does changing b[0] change a[0]?
3. Does a == b compare array contents?

Pass by value with an array reference

```
int number = 1;
int[] values = {1};
change(number, values);
System.out.println(number);
System.out.println(values[0]);

static void change(int number, int[] values) {
    number = 1001;
    values[0] = 5555;
}
```

The parameter receives a copy of the array reference.

Reassigning the parameter reference

```
int[] values = {1, 2};
replace(values);
System.out.println(Arrays.toString(values));

static void replace(int[] values) {
    values = new int[]{9, 9};
}
```

Reassigning the local parameter leaves the caller reference unchanged.

Quiz 5: mutation and reassignment

A method receives int[] values.

1. Can values[0] = 7 affect the caller?
2. Can values = new int[]{7} replace
   the caller's array reference?
3. Does Java pass arrays by reference?

Anonymous arrays as arguments

```
printArray(new int[]{3, 1, 2, 6});

static void printArray(int[] values) {
    for (int value : values) {
        System.out.println(value);
    }
}
```

A call can create an array without first naming a local reference.

Returning a new array

```
static int[] reverse(int[] values) {
    int[] result = new int[values.length];
    for (int i = 0; i < values.length; i++) {
        result[i] = values[values.length - 1 - i];
    }
    return result;
}
```

reverse returns a new reference and preserves the original elements.

Case study: counting random letters

```
Random random = new Random(7);
char[] letters = new char[100];
for (int i = 0; i < letters.length; i++) {
    letters[i] = (char) ('a' + random.nextInt(26));
}
int[] counts = countLetters(letters);
for (int i = 0; i < counts.length; i++) {
    System.out.println((char) ('a' + i) + ": " + counts[i]);
}
```

A fixed seed lets the class reproduce the same 100 letters.

Mapping a letter to a frequency index

```
static int[] countLetters(char[] letters) {
    int[] counts = new int[26];
    for (char letter : letters) {
        counts[letter - 'a']++;
    }
    return counts;
}
```

'a' uses index 0. 'z' uses index 25.

Variable-length argument lists

```
static int sum(int... values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}

System.out.println(sum());
System.out.println(sum(2, 4, 6));
System.out.println(sum(new int[]{5, 7}));
```

A varargs parameter acts as an array inside the method.

Quiz 6: array methods and varargs

1. What does reverse(new int[0]) return?
2. What does sum() return in our example?
3. Can a varargs parameter come first when another parameter follows it?
4. Can sum(int[]) overload sum(int...)?

Linear search

```
static int linearSearch(int[] values, int key) {
    for (int i = 0; i < values.length; i++) {
        if (values[i] == key) return i;
    }
    return -1;
}
```

Return the first matching index, or -1 when no value matches.

Tracing a linear search

Array: [6, 4, 1, 9, 7, 3, 2, 8]
Key: 3

Compare indexes 0, 1, 2, 3, 4, then 5.
Index 5 contains 3, so return 5.

Discussion: What changes if the key is absent?

Binary search needs sorted input

For an ascending array:
Compare the key with the middle element.
Keep only the half that could contain the key.
Repeat until found or the interval is empty.

Discussion: What fails on an unsorted array?

Binary search implementation

```
static int binarySearch(int[] values, int key) {
    int low = 0, high = values.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (values[mid] == key) return mid;
        if (values[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

This teaching method returns -1 for a missing key.

Tracing binary search

Array: [2, 4, 7, 10, 11, 45, 50]
Key: 11

low 0, high 6, mid 3: value 10
low 4, high 6, mid 5: value 45
low 4, high 4, mid 4: value 11
Return index 4

Quiz 7: choosing a search

1. Which search can use unsorted data?
2. Which precondition does binary search need?
3. Does our binary search return a value
   or an index when it finds a match?
4. Must it return the first duplicate?

Selection sort

At position i, find the smallest value
in the unsorted suffix and swap it into i.

Example: [4, 2, 3, 1]
After pass 1: [1, 2, 3, 4]

The sorted prefix grows by one each pass.

Selection sort: placing the next value

```
static void selectionSort(int[] values) {
    for (int i = 0; i < values.length - 1; i++) {
        int smallest = indexOfSmallest(values, i);
        int temp = values[i];
        values[i] = values[smallest];
        values[smallest] = temp;
    }
}
```

The method mutates the array in place.

Selection sort: finding a suffix minimum

```
static int indexOfSmallest(int[] values, int start) {
    int smallest = start;
    for (int j = start + 1; j < values.length; j++) {
        if (values[j] < values[smallest]) smallest = j;
    }
    return smallest;
}
```

The helper returns the smallest value's index in the suffix.

The java.util.Arrays utilities

```
int[] values = {6, 4, 1, 2};
Arrays.sort(values);
int[] copy = Arrays.copyOf(values, values.length);
System.out.println(Arrays.toString(values));
System.out.println(Arrays.equals(values, copy));
System.out.println(values == copy);
```

sort mutates. toString formats. equals compares contents.

Arrays.binarySearch and insertion points

```
int[] values = {2, 4, 7, 10};
System.out.println(Arrays.binarySearch(values, 7));
int result = Arrays.binarySearch(values, 8);
System.out.println(result);
System.out.println(-result - 1);
```

Missing key: result = -insertionPoint - 1.

Quiz 8: sorting and library search

1. Does Arrays.sort create a separate array?
2. Does Arrays.equals compare contents?
3. For [2, 4, 7, 10], what does Arrays.binarySearch(values, 5) return?
4. What does its result -1 mean?

Command-line arguments

```
System.out.println("Count: " + args.length);
for (int i = 0; i < args.length; i++) {
    System.out.println(i + ": " + args[i]);
}
```

Run: java CommandLineDemo hello 42

Calling main as a static method

```
CommandLineDemo.main(new String[]{"hello", "42"});
```

main can receive a String array through an ordinary method call.

Case study: command-line calculator

```
if (args.length != 3) {
    System.out.println("Usage: java Calculator 2 + 3");
    return;
}
try {
    int left = Integer.parseInt(args[0]);
    int right = Integer.parseInt(args[2]);
    System.out.println(calculate(left, args[1], right));
} catch (IllegalArgumentException | ArithmeticException ex) {
    System.out.println("Error: " + ex.getMessage());
}
```

Run: java Calculator 2 + 3

Calculator operation method

```
static int calculate(int a, String op, int b) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        case "/": return a / b;
        default:
            throw new IllegalArgumentException(
                "Unknown operator");
    }
}
```

Integer division truncates toward zero.

Quiz 9: calculator arguments

1. How many elements does args contain for java Calculator 7 / 2?
2. What result does that command print?
3. What happens with no arguments?
4. What happens with a decimal operand?

Exit exercise: values above a limit

Design countAbove(int[] values, int limit).
Return how many elements exceed limit.

Predict the result for {2, 7, 7, 1} and limit 5.
What should an empty array return?
Does equality count?

Exit exercise: one possible solution

```
static int countAbove(int[] values, int limit) {
    int count = 0;
    for (int value : values) {
        if (value > limit) count++;
    }
    return count;
}
```

Tests: {2, 7, 7, 1} with limit 5 gives 2. Empty input gives 0.

Chapter review

Indexes run from 0 through length - 1.
Assignment can create an alias.
Copies need separate array storage.
Methods receive copied reference values.
Binary search needs sorted data.
Contracts define valid inputs and results.